

Arctangent Implementation

(Joint angle Calculation Via Trigonometric Vector Analysis)

Core Mathematical Utility:

```
1 // Calculate Pose detection angles
2 // To calculate angles between hip, knee, and ankles for Squats
3 import 'dart:math' as math;
4 import 'package:google_mlkit_pose_detection/google_mlkit_pose_detection.dart';
5
6 class PoseMath {
7   // Function to calculate angle between Hip(A), Knee(B), and Ankle(C)
8   static double getAngle(PoseLandmark firstPoint, PoseLandmark midPoint, PoseLandmark lastPoint) {
9     // Using atan2 math function (Arctangent)
10    // the logic is to find the angle of line from midPoint to firstPoint and subtract it from the angle of the line from midp
11    var result = (math.atan2(lastPoint.y - midPoint.y, lastPoint.x - midPoint.x) -
12                  math.atan2(firstPoint.y - midPoint.y, firstPoint.x - midPoint.x))
13                  .abs() * (180 / math.pi); // Convert radians to degrees
14
15    // Ensure the angle is always under 180 degree
16    if (result > 180) {
17      result = 360 - result;
18    }
19    return result;
20  }
21 }
```

Figure 1: Core Logic

This utility function is the mathematical engine of the KasRrat application which utilizes **atan2 (two-argument arctangent) function** to compute the angle at a specific joint. Unlike the standard atan function, atan2 handles all four quadrants and prevents 'division by zero' errors. This ensures that the app can accurately track movement even if the user is standing at an angle to the camera.

1. Squats Implementation

```
// Knee angles for both sides
double leftKneeAngle = PoseMath.getAngle(leftHip, leftKnee, leftAnkle);
double rightKneeAngle = PoseMath.getAngle(rightHip, rightKnee, rightAnkle);

// Hip angles for all sides (Shoulder -> Hip -> Knee)
double leftHipAngle = PoseMath.getAngle(leftShoulder, leftHip, leftKnee);
double rightHipAngle = PoseMath.getAngle(rightShoulder, rightHip, rightKnee);

// Back Straightness Check
double torsoAngle = leftShoulder.likelihood > rightShoulder.likelihood
    ? PoseMath.getAngle(
        PoseLandmark(type: PoseLandmarkType.leftShoulder, x: leftShoulder.x, y: leftShoulder.y - 100, z: 0, likelihood: 1),
        leftHip, leftShoulder)
    : PoseMath.getAngle(
        PoseLandmark(type: PoseLandmarkType.rightShoulder, x: rightShoulder.x, y: rightShoulder.y - 100, z: 0, likelihood: 1),
        rightHip, rightShoulder);

if (torsoAngle > 30) {
    feedback = "Keep your back straight!";
    if (!hasFormError) onFormError(); // Trigger voice once
    hasFormError = true;
    repHadError = true; // Persistent memory that the back was bent during the rep
    errorsFound.add("Back Bending"); // Record the error type
} else {
    hasFormError = false;
}

// If one knee is bent significantly more than the other, then its considered a lunge
if (kneeDiff > 40 && (leftKneeAngle < 130 || rightKneeAngle < 130)) {
    feedback = "Perform a Squat, thats a Lunge!";
    if (!hasFormError) onFormError(); // Trigger voice once
    hasFormError = true;
    repHadError = true;
    errorsFound.add("Form Issues"); // Record the error type
    return;
}

// Bilateral Check
// uses && for both knees and both hips
if (leftKneeAngle < 100 && rightKneeAngle < 100 && leftHipAngle < 120 && rightHipAngle < 120) {
    isDown = true;
    depthAchieved = true;
    if (!hasFormError) feedback = "Now Stand Up!";
}

// check if one leg is deep but the other is not
else if ((leftKneeAngle < 100 && rightKneeAngle > 115) || (rightKneeAngle < 100 && leftKneeAngle > 115)) {
    feedback = "Balance your weight on both legs!";
    if (!hasFormError) onFormError(); // Trigger voice once
    hasFormError = true;
    repHadError = true; // if yes block the rep
    errorsFound.add("Form Issues");
}

// if the squat depth is not enough
else if (leftKneeAngle < 140 || rightKneeAngle < 140) {
    isDown = true;
    if (!depthAchieved) {
        hasFormError = true;
        feedback = "Go Lower!";
    }
}
}
```

Figure 2: Squat math logic

Squat detection logic monitors 3 key landmarks (Hips → Knee → Ankle) and when the computed angle between these 3 points fall below 90 degrees, the system registers a “Down” state. The repetition is only incremented when the user returns to a standing position (angle > 160), ensuring the user completes the full range of motion.

2. Push-up Implementation

```
// if the hip is too high or too low, marking it as error
// In a pushup, a body angle < 160 usually means the hips are sagging or piking
if (bodyAngle < 160) {
    feedback = "Keep your body straight!";
    if (!hasFormError) onFormError(); // Trigger voice once
    hasFormError = true;
    repHadError = true; // Persistent memory that the body was not straight
    errorsFound.add("Back Bending"); // Record the error type
} else {
    hasFormError = false;
}

// Down position detection
if (currentElbowAngle < 90) {
    isDown = true;
    depthAchieved = true;
    if (!hasFormError) feedback = "Push Up!";
}
// If partially down but not deep enough
else if (currentElbowAngle < 130) {
    isDown = true;
    if (!depthAchieved) {
        hasFormError = true;
        feedback = "Go Lower!";
    }
}
}
```

```
// Up position detection
if (currentElbowAngle > 160) {
    if (isDown) {
        totalAttempts++; // to count every pushup attempt

        // Rep is counted only if depth was achieved AND no form errors occurred
        if (depthAchieved && !repHadError) {
            reps++;
            goodReps++;
            onRepCount(); // rep count audio feedback
            feedback = "Good Rep!";
            hasFormError = false;
        } else if (repHadError) {
            // If form was bad, rep is not counted
            onFormError(); // Error audio feedback
            feedback = "Form was off, keep your body straight";
            hasFormError = true;
        } else if (!depthAchieved) {
            // If pushup depth was not enough, rep is not counted
            onFormError(); // Error audio feedback
            feedback = "Didn't go low enough, try again go deeper";
            hasFormError = true;
            errorsFound.add("Shallow Depth");
        }
    }
}
```

Figure 3: Push-up math logic

Push-up detection checks for a 70-degree angle at the elbow to verify a successful descent which also includes stability check where the Y-coordinates of the hip and shoulder are compared to ensure the user is maintaining a straight back, preventing lower-back injury.

3. Lateral-Raises Implementation

```
// User must raise both arms for the rep to be counted
double armDiff = (leftShoulderAngle - rightShoulderAngle).abs();
if (armDiff > 25 && (leftShoulderAngle > 60 || rightShoulderAngle > 60)) {
    feedback = "Raise both arms evenly Together!";
    if (!hasFormError) onFormError();
    hasFormError = true;
    repHadError = true;
    errorsFound.add("Asymmetric Raise");
} else {
    hasFormError = false;
}

// Top position --> arms at shoulder level
if (leftShoulderAngle > 80 && rightShoulderAngle > 80) {
    isUp = true;
    peakAchieved = true;
    if (!hasFormError) feedback = "Now lower them slowly";
}

// Middle zone
else if (leftShoulderAngle > 50 || rightShoulderAngle > 50) {
    isUp = true;
    if (!peakAchieved) {
        if (!hasFormError) feedback = "Raise to shoulder level!";
    }
}

// Bottom position --> arms down
if (leftShoulderAngle < 30 && rightShoulderAngle < 30) {
    if (isUp) {
        totalAttempts++;

        if (peakAchieved && !repHadError) {
            reps++;
            goodReps++;
            onRepCount();
            feedback = "Good Rep!";
            hasFormError = false;
        } else if (repHadError) {
            onFormError();
            feedback = "Form was off, face the camera and be even";
            hasFormError = true;
        } else if (!peakAchieved) {
            onFormError();
            feedback = "Didn't raise high enough";
            hasFormError = true;
            errorsFound.add("Shallow Raise");
        }
    }
}
```

Figure 4: Lateral-Raise math logic

For lateral raises the rep logic is defined by three specific angular zones, the peak is reached when both arms reach an abduction angle of at least 80 degrees what should be parallel to the floor. A rep is only recorded if the user reaches the peak angle without triggering the symmetry or orientation errors.

4. Bicep-curl Implementation

```
// If the elbow is more than 40px ahead of the shoulder → swinging cheat detected
// (In ML Kit coordinates, a larger X value = further to the right in the raw image)
bool isSwinging = (elbowX - shoulderX).abs() > 40 && (leftElbowAngle < 100 || rightElbowAngle < 100);

if (isSwinging) {
    feedback = "Keep your elbow still, don't swing!";
    if (!hasFormError) onFormError(); // Trigger tts for form error
    hasFormError = true;
    repHadError = true; // Persistent memory to save that the elbow swung during the rep
    errorsFound.add("Elbow Swinging"); // Record the error type
} else {
    hasFormError = false;
}

// Rep is only valid if Both arms are curled together
bool bothAtPeak = leftElbowAngle < 60 && rightElbowAngle < 60;
bool bothDown = leftElbowAngle > 150 && rightElbowAngle > 150;

// Check for asymmetry curl
if ((leftElbowAngle < 60 && rightElbowAngle > 100) || (rightElbowAngle < 60 && leftElbowAngle > 100)) {
    feedback = "Curl both arms together!";
    if (!hasFormError) onSymmetryError(); // Trigger tts for single arm curl error
    hasFormError = true;
    repHadError = true;
    errorsFound.add("Form Issues");
}
```

```
// top position detection (curl) --> Elbow < 60
if (bothAtPeak) {
    isUp = true;
    peakAchieved = true;
    if (!hasFormError) feedback = "Lower it down!";
}

// Partially curled but not high enough
else if (leftElbowAngle < 90 || rightElbowAngle < 90) {
    isUp = true;
    if (!peakAchieved) {
        hasFormError = true;
        feedback = "Curl Higher!";
    }
}

// Bottom position detection (straight) --> Elbow > 150
if (bothDown) {
    if (isUp) {
        totalAttempts++; // counting every lowering as an attempt

        // Rep is counted only if peak was achieved AND no form errors occurred
        if (peakAchieved && !repHadError) {
            reps++;
            goodReps++;
            onRepCount(); // rep count audio feedback
            feedback = "Good Rep!";
            hasFormError = false;
        } else if (repHadError) {
            // If form was bad (elbow swinging), rep is not counted
            onFormError(); // Error audio feedback
            feedback = "Rep not counted, keep your elbow in a fixed position";
            hasFormError = true;
        } else if (!peakAchieved) {
            // If curl height wasn't enough, rep is not counted
            onFormError(); // Error audio feedback
            feedback = "You didn't curl high enough, try again";
            hasFormError = true;
            errorsFound.add("Shallow Curl");
        }
    }
}
```

Figure 5: Bicep-curl math logic

For Bicep curls the landmarks for shoulder, elbow and wrist are tracked. To prevent cheat (half) reps, the logic requires the elbow angle to reach a contraction of less than 35 degrees.